MINI PROJECT

Name: Riza sajjad shaikh
Roll no : AI2094
Topic : Student management system

Code:

```
import tkinter as tk
from tkinter import ttk, messagebox
import sqlite3

# ================= DATABASE =================
db = sqlite3.connect("smart_students.db")
cur = db.cursor()

cur.execute("""
CREATE TABLE IF NOT EXISTS students (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    roll TEXT UNIQUE NOT NULL,
    name TEXT NOT NULL,
    course TEXT NOT NULL,
    marks REAL NOT NULL,
    attendance REAL NOT NULL
)
""")

# Add address column automatically to an old database
cur.execute("PRAGMA table_info(students)")
columns_in_db = [col[1] for col in cur.fetchall()]

if "address" not in columns_in_db:
    cur.execute(
        "ALTER TABLE students ADD COLUMN address TEXT NOT NULL DEFAULT ''"
    )

db.commit()

# ================= COLORS =================
BG = "#F5F3FF"
PURPLE = "#6D28D9"
DARK = "#24124D"
WHITE = "#FFFFFF"
TEXT = "#292044"
MUTED = "#807795"
GREEN = "#059669"
ORANGE = "#EA580C"
RED = "#DC2626"

COURSES = [
    "CSE",
    "AI & ML",
    "Civil Engineering",
    "Mechanical Engineering",
```

```
    "Electronics Engineering",
    "Electrical Engineering"
]

# ================= WINDOW =================
root = tk.Tk()
root.title("Smart Student | Management System")
root.geometry("1200x760")
root.minsize(1000, 650)
root.configure(bg=BG)

style = ttk.Style()
style.theme_use("clam")

style.configure(
    "Treeview",
    background=WHITE,
    foreground=TEXT,
    rowheight=30,
    fieldbackground=WHITE,
    font=("Arial", 9)
)

style.configure(
    "Treeview.Heading",
    background=DARK,
    foreground=WHITE,
    padding=8,
    font=("Arial", 9, "bold")
)

style.map(
    "Treeview",
    background=[("selected", "#DDD6FE")]
)

# ================= VARIABLES =================
roll_var = tk.StringVar()
name_var = tk.StringVar()
course_var = tk.StringVar(value=COURSES[0])
address_var = tk.StringVar()
marks_var = tk.StringVar()
attendance_var = tk.StringVar(value="100")
selected_id = tk.StringVar()
search_var = tk.StringVar()

# ================= GRADE FUNCTION =================
def grade(marks):
    if marks >= 90:
        return "A+"
    elif marks >= 80:
        return "A"
    elif marks >= 70:
        return "B"
    elif marks >= 60:
```

```
        return "C"
    elif marks >= 40:
        return "D"
    return "Fail"


# ================= CLEAR FORM =================
def clear_form():
    roll_var.set("")
    name_var.set("")
    course_var.set(COURSES[0])
    address_var.set("")
    marks_var.set("")
    attendance_var.set("100")
    selected_id.set("")

    if table.selection():
        table.selection_remove(table.selection())


# ================= LOAD SELECTED STUDENT =================
def load_form(event=None):
    chosen = table.selection()

    if not chosen:
        return

    row = table.item(chosen[0], "values")

    selected_id.set(str(row[0]))
    roll_var.set(row[1])
    name_var.set(row[2])
    course_var.set(row[3])
    address_var.set(row[4])
    marks_var.set(row[5])
    attendance_var.set(row[6].replace("%", ""))


# ================= REFRESH TABLE =================
def refresh():
    for item in table.get_children():
        table.delete(item)

    keyword = search_var.get().strip()

    cur.execute("""
        SELECT id, roll, name, course, address, marks, attendance
        FROM students
        WHERE roll LIKE ?
          OR name LIKE ?
          OR course LIKE ?
          OR address LIKE ?
        ORDER BY id DESC
    """, (
        f"%{keyword}%",
```

```
            f"%{keyword}%",
            f"%{keyword}%",
            f"%{keyword}%"
        ))

        records = cur.fetchall()

        for row in records:
            table.insert("", "end", values=(
                row[0],
                row[1],
                row[2],
                row[3],
                row[4],
                f"{row[5]:g}",
                f"{row[6]:g}%",
                grade(row[5])
            ))

        cur.execute("""
            SELECT COUNT(*), AVG(marks), COUNT(DISTINCT course)
            FROM students
        """)

        total, average, courses = cur.fetchone()

        total_value.config(text=str(total))
        course_value.config(text=str(courses))

        average_value.config(
            text=f"{average:.1f}" if average is not None else "0.0"
        )

        result_label.config(
            text=f"{len(records)} student(s) displayed"
        )


    # ================== GET FORM DATA ==================
    def get_data():
        roll = roll_var.get().strip()
        name = name_var.get().strip()
        course = course_var.get().strip()
        address = address_var.get().strip()

        if not roll or not name or not course or not address:
            messagebox.showwarning(
                "Missing Details",
                "Please enter Roll Number, Name, Course and Address."
            )
            return None

        if course not in COURSES:
            messagebox.showerror(
                "Invalid Course",
```

```
            "Please select a valid course."
        )
        return None

    try:
        marks = float(marks_var.get())
        attendance = float(attendance_var.get())

        if not 0 <= marks <= 100:
            raise ValueError("Marks must be between 0 and 100.")

        if not 0 <= attendance <= 100:
            raise ValueError(
                "Attendance must be between 0 and 100."
            )

    except ValueError as error:
        messagebox.showerror(
            "Invalid Input",
            str(error) or "Please enter valid numeric values."
        )
        return None

    return roll, name, course, address, marks, attendance


# ================== ADD STUDENT ==================
def add_student():
    data = get_data()

    if data is None:
        return

    try:
        cur.execute("""
            INSERT INTO students
            (roll, name, course, address, marks, attendance)
            VALUES (?, ?, ?, ?, ?, ?)
        """, data)

        db.commit()
        refresh()
        clear_form()

        messagebox.showinfo(
            "Success",
            "Student added successfully!"
        )

    except sqlite3.IntegrityError:
        messagebox.showerror(
            "Duplicate Roll Number",
            "This Roll Number already exists."
        )
```

```
# ================= UPDATE STUDENT =================
def update_student():
    if not selected_id.get():
        messagebox.showwarning(
            "Select Student",
            "Select a student from the table first."
        )
        return

    data = get_data()

    if data is None:
        return

    try:
        cur.execute("""
            UPDATE students
            SET roll=?, name=?, course=?, address=?,
                marks=?, attendance=?
            WHERE id=?
        """, (*data, selected_id.get()))

        db.commit()
        refresh()
        clear_form()

        messagebox.showinfo(
            "Updated",
            "Student record updated successfully!"
        )

    except sqlite3.IntegrityError:
        messagebox.showerror(
            "Duplicate Roll Number",
            "Another student already uses this Roll Number."
        )


# ================= DELETE STUDENT =================
def delete_student():
    if not selected_id.get():
        messagebox.showwarning(
            "Select Student",
            "Select a student from the table first."
        )
        return

    confirm = messagebox.askyesno(
        "Confirm Delete",
        "Are you sure you want to delete this student?"
    )

    if confirm:
        cur.execute(
```

```
            "DELETE FROM students WHERE id=?",
            (selected_id.get(),)
        )

        db.commit()
        refresh()
        clear_form()

        messagebox.showinfo(
            "Deleted",
            "Student record deleted successfully."
        )


# ================== SHOW ALL STUDENTS ==================
def show_all():
    search_var.set("")
    refresh()


# ================== BUTTON DESIGN ==================
def make_button(parent, text, color, command):
    return tk.Button(
        parent,
        text=text,
        command=command,
        bg=color,
        fg=WHITE,
        activebackground=color,
        activeforeground=WHITE,
        relief="flat",
        cursor="hand2",
        font=("Arial", 9, "bold"),
        padx=10,
        pady=7
    )


# ================== REPORT WINDOW ==================
def open_reports():
    report_win = tk.Toplevel(root)
    report_win.title("Student Reports")
    report_win.geometry("1100x620")
    report_win.minsize(850, 500)
    report_win.configure(bg=BG)

    tk.Label(
        report_win,
        text="Student Reports",
        bg=BG,
        fg=TEXT,
        font=("Arial", 22, "bold")
    ).pack(anchor="w", padx=20, pady=(18, 3))

    tk.Label(
```

```
    report_win,
    text="View student records by address, branch or marks.",
    bg=BG,
    fg=MUTED,
    font=("Arial", 10)
).pack(anchor="w", padx=20, pady=(0, 14))

# Report controls
controls = tk.Frame(
    report_win,
    bg=WHITE,
    padx=12,
    pady=12
)
controls.pack(fill="x", padx=20, pady=(0, 12))

tk.Label(
    controls,
    text="Report Type:",
    bg=WHITE,
    fg=TEXT,
    font=("Arial", 10, "bold")
).pack(side="left", padx=(0, 8))

report_type = tk.StringVar(value="All Students")

report_combo = ttk.Combobox(
    controls,
    textvariable=report_type,
    values=[
        "All Students",
        "Address-wise",
        "Branch-wise",
        "Marks-wise"
    ],
    state="readonly",
    width=18
)
report_combo.pack(side="left", padx=(0, 12))

filter_var = tk.StringVar()

filter_entry = tk.Entry(
    controls,
    textvariable=filter_var,
    font=("Arial", 10),
    relief="solid",
    bd=1,
    width=23
)
filter_entry.pack(side="left", ipady=5, padx=(0, 8))

hint_label = tk.Label(
    controls,
    text="Optional filter",
```

```
    bg=WHITE,
    fg=MUTED,
    font=("Arial", 9)
)
hint_label.pack(side="left", padx=(0, 10))

# Report table
report_columns = (
    "ID", "Roll No", "Name", "Course",
    "Address", "Marks", "Attendance", "Grade"
)

report_table = ttk.Treeview(
    report_win,
    columns=report_columns,
    show="headings",
    selectmode="browse"
)

widths = [40, 75, 130, 145, 220, 65, 90, 65]

for col, width in zip(report_columns, widths):
    report_table.heading(col, text=col)
    report_table.column(
        col,
        width=width,
        minwidth=55,
        anchor="center"
    )

report_scroll_y = ttk.Scrollbar(
    report_win,
    orient="vertical",
    command=report_table.yview
)

report_scroll_x = ttk.Scrollbar(
    report_win,
    orient="horizontal",
    command=report_table.xview
)

report_table.configure(
    yscrollcommand=report_scroll_y.set,
    xscrollcommand=report_scroll_x.set
)

report_scroll_y.pack(
    side="right",
    fill="y",
    padx=(0, 20)
)

report_table.pack(
    fill="both",
```

```
    expand=True,
    padx=(20, 0)
)

report_scroll_x.pack(
    fill="x",
    padx=20
)

report_status = tk.Label(
    report_win,
    text="",
    bg=BG,
    fg=MUTED,
    font=("Arial", 9)
)
report_status.pack(anchor="w", padx=20, pady=10)

# Change filter instructions
def update_hint(event=None):
    kind = report_type.get()

    if kind == "Address-wise":
        hint_label.config(text="Enter address (optional)")
    elif kind == "Branch-wise":
        hint_label.config(text="Enter branch (optional)")
    elif kind == "Marks-wise":
        hint_label.config(text="Minimum marks, e.g. 75")
    else:
        hint_label.config(text="Name, roll or course (optional)")

# Generate selected report
def generate_report():
    for item in report_table.get_children():
        report_table.delete(item)

    kind = report_type.get()
    keyword = filter_var.get().strip()

    sql = """
        SELECT id, roll, name, course, address, marks, attendance
        FROM students
    """

    params = []
    order = " ORDER BY id DESC"

    if kind == "Address-wise":
        if keyword:
            sql += " WHERE address LIKE ?"
            params.append(f"%{keyword}%")

        order = """
            ORDER BY address COLLATE NOCASE,
                     name COLLATE NOCASE
```

```
        """

    elif kind == "Branch-wise":
        if keyword:
            sql += " WHERE course LIKE ?"
            params.append(f"%{keyword}%")

        order = """
            ORDER BY course COLLATE NOCASE,
                     name COLLATE NOCASE
        """

    elif kind == "Marks-wise":
        if keyword:
            try:
                minimum_marks = float(keyword)

                if not 0 <= minimum_marks <= 100:
                    raise ValueError

            except ValueError:
                messagebox.showerror(
                    "Invalid Marks",
                    "Enter marks between 0 and 100, e.g. 75."
                )
                return

            sql += " WHERE marks >= ?"
            params.append(minimum_marks)

        order = " ORDER BY marks DESC, name COLLATE NOCASE"

    else:
        if keyword:
            sql += """
                WHERE roll LIKE ?
                  OR name LIKE ?
                  OR course LIKE ?
                  OR address LIKE ?
            """
            params.extend([f"%{keyword}%"] * 4)

    cur.execute(sql + order, params)
    records = cur.fetchall()

    for row in records:
        report_table.insert("", "end", values=(
            row[0],
            row[1],
            row[2],
            row[3],
            row[4],
            f"{row[5]:g}",
            f"{row[6]:g}%",
            grade(row[5])
```

```
            ))

        if kind == "Marks-wise" and keyword:
            report_status.config(
                text=f"{len(records)} student(s) with marks >= {keyword}"
            )
        elif kind == "Address-wise":
            report_status.config(
                text=f"{len(records)} student(s), sorted by address"
            )
        elif kind == "Branch-wise":
            report_status.config(
                text=f"{len(records)} student(s), sorted by branch"
            )
        else:
            report_status.config(
                text=f"{len(records)} student(s) in this report"
            )

    make_button(
        controls,
        "Generate Report",
        PURPLE,
        generate_report
    ).pack(side="left", padx=4)

    def reset_report():
        filter_var.set("")
        report_type.set("All Students")
        update_hint()
        generate_report()

    make_button(
        controls,
        "Show All",
        "#64748B",
        reset_report
    ).pack(side="left", padx=4)

    report_combo.bind("<<ComboboxSelected>>", update_hint)
    filter_entry.bind(
        "<Return>",
        lambda event: generate_report()
    )

    update_hint()
    generate_report()


# ================= SIDEBAR =================
sidebar = tk.Frame(root, bg=DARK, width=210)
sidebar.pack(side="left", fill="y")
sidebar.pack_propagate(False)

tk.Label(
```

```
    sidebar,
    text="✦ SMART STUDENT",
    bg=DARK,
    fg=WHITE,
    font=("Arial", 17, "bold")
).pack(pady=(32, 5))

tk.Label(
    sidebar,
    text="MANAGEMENT PORTAL",
    bg=DARK,
    fg="#C4B5FD",
    font=("Arial", 9, "bold")
).pack(pady=(0, 36))

tk.Label(
    sidebar,
    text="▦    Dashboard",
    bg=PURPLE,
    fg=WHITE,
    font=("Arial", 12, "bold"),
    anchor="w",
    padx=18,
    pady=13
).pack(fill="x", padx=12)

tk.Label(
    sidebar,
    text=" ♙   Students",
    bg=DARK,
    fg="#DDD6FE",
    font=("Arial", 12),
    anchor="w",
    padx=18,
    pady=13
).pack(fill="x", padx=12)

reports_nav = tk.Label(
    sidebar,
    text="▤    Reports",
    bg=DARK,
    fg="#DDD6FE",
    font=("Arial", 12),
    anchor="w",
    padx=18,
    pady=13,
    cursor="hand2"
)
reports_nav.pack(fill="x", padx=12)

reports_nav.bind(
    "<Button-1>",
    lambda event: open_reports()
)
```

```
reports_nav.bind(
    "<Enter>",
    lambda event: reports_nav.config(bg=PURPLE, fg=WHITE)
)

reports_nav.bind(
    "<Leave>",
    lambda event: reports_nav.config(bg=DARK, fg="#DDD6FE")
)

tk.Label(
    sidebar,
    text="LEARN • TRACK • GROW\n\n"
         "Simple student records,\nall in one place.",
    bg=DARK,
    fg="#A78BFA",
    font=("Arial", 10),
    justify="left"
).pack(side="bottom", anchor="w", padx=20, pady=25)


# ================== MAIN AREA ==================
main = tk.Frame(root, bg=BG)
main.pack(
    side="left",
    fill="both",
    expand=True,
    padx=20,
    pady=18
)

header = tk.Frame(main, bg=BG)
header.pack(fill="x")

tk.Label(
    header,
    text="Dashboard Overview",
    bg=BG,
    fg=TEXT,
    font=("Arial", 23, "bold")
).pack(side="left")

tk.Label(
    header,
    text="Welcome, Admin",
    bg=BG,
    fg=MUTED,
    font=("Arial", 10)
).pack(side="right", pady=10)

tk.Label(
    main,
    text="Your students. Your progress. One smart dashboard.",
    bg=BG,
    fg=MUTED,
```

```
    font=("Arial", 10)
).pack(anchor="w", pady=(4, 17))


# ================= STATISTICS CARDS =================
stats = tk.Frame(main, bg=BG)
stats.pack(fill="x", pady=(0, 17))

for i in range(3):
    stats.columnconfigure(i, weight=1)


def make_stat(col, title, color, initial):
    card = tk.Frame(
        stats,
        bg=WHITE,
        padx=15,
        pady=14,
        highlightbackground="#E9E2F8",
        highlightthickness=1
    )

    card.grid(
        row=0,
        column=col,
        sticky="ew",
        padx=(0 if col == 0 else 7, 0)
    )

    tk.Label(
        card,
        text=title,
        bg=WHITE,
        fg=MUTED,
        font=("Arial", 10, "bold")
    ).pack(anchor="w")

    value = tk.Label(
        card,
        text=initial,
        bg=WHITE,
        fg=color,
        font=("Arial", 24, "bold")
    )

    value.pack(anchor="w", pady=(5, 0))
    return value


total_value = make_stat(0, "TOTAL STUDENTS", PURPLE, "0")
course_value = make_stat(1, "ACTIVE COURSES", GREEN, "0")
average_value = make_stat(2, "AVERAGE MARKS", ORANGE, "0.0")


# ================= STUDENT FORM =================
```

```
form = tk.Frame(
    main,
    bg=WHITE,
    padx=14,
    pady=10,
    highlightbackground="#E9E2F8",
    highlightthickness=1
)
form.pack(fill="x", pady=(0, 14))

tk.Label(
    form,
    text="✚  Student Details",
    bg=WHITE,
    fg=TEXT,
    font=("Arial", 14, "bold")
).grid(
    row=0,
    column=0,
    columnspan=4,
    sticky="w",
    pady=(0, 7)
)

for i in range(4):
    form.columnconfigure(i, weight=1)


def add_field(title, variable, row, col, values=None):
    tk.Label(
        form,
        text=title,
        bg=WHITE,
        fg=MUTED,
        font=("Arial", 9, "bold")
    ).grid(
        row=row,
        column=col,
        sticky="w",
        padx=5,
        pady=(2, 3)
    )

    if values:
        widget = ttk.Combobox(
            form,
            textvariable=variable,
            values=values,
            state="readonly",
            font=("Arial", 10)
        )
    else:
        widget = tk.Entry(
            form,
            textvariable=variable,
```

```
            font=("Arial", 10),
            relief="solid",
            bd=1
        )

    widget.grid(
        row=row + 1,
        column=col,
        sticky="ew",
        padx=5,
        pady=(0, 6),
        ipady=4
    )

    return widget


add_field("Roll Number *", roll_var, 1, 0)
add_field("Student Name *", name_var, 1, 1)
add_field("Course / Branch *", course_var, 1, 2, COURSES)
add_field("Marks (0-100) *", marks_var, 1, 3)
add_field("Address *", address_var, 3, 0)
add_field("Attendance % *", attendance_var, 3, 1)

buttons = tk.Frame(form, bg=WHITE)
buttons.grid(
    row=4,
    column=2,
    columnspan=2,
    sticky="e",
    pady=3
)

make_button(buttons, "+ Add", PURPLE, add_student).pack(
    side="left", padx=3
)

make_button(buttons, "Update", GREEN, update_student).pack(
    side="left", padx=3
)

make_button(buttons, "Delete", RED, delete_student).pack(
    side="left", padx=3
)

make_button(buttons, "Clear", "#64748B", clear_form).pack(
    side="left", padx=3
)


# ================== STUDENT TABLE ==================
records_box = tk.Frame(
    main,
    bg=WHITE,
    padx=10,
```

```
    pady=10,
    highlightbackground="#E9E2F8",
    highlightthickness=1
)
records_box.pack(fill="both", expand=True)

bar = tk.Frame(records_box, bg=WHITE)
bar.pack(fill="x", pady=(0, 8))

tk.Label(
    bar,
    text="Student Directory",
    bg=WHITE,
    fg=TEXT,
    font=("Arial", 14, "bold")
).pack(side="left")

search_entry = tk.Entry(
    bar,
    textvariable=search_var,
    font=("Arial", 10),
    relief="solid",
    bd=1,
    width=18
)
search_entry.pack(side="right", padx=(5, 0), ipady=4)

make_button(bar, "Search", PURPLE, refresh).pack(
    side="right", padx=4
)

make_button(bar, "Show All", "#64748B", show_all).pack(
    side="right", padx=4
)

make_button(bar, "Reports", GREEN, open_reports).pack(
    side="right", padx=4
)

columns = (
    "ID", "Roll No", "Name", "Course",
    "Address", "Marks", "Attendance", "Grade"
)

table = ttk.Treeview(
    records_box,
    columns=columns,
    show="headings",
    selectmode="browse"
)

widths = [35, 70, 115, 125, 170, 55, 80, 55]

for col, width in zip(columns, widths):
    table.heading(col, text=col)
```

```
    table.column(
        col,
        width=width,
        minwidth=45,
        anchor="center"
    )

scroll = ttk.Scrollbar(
    records_box,
    orient="vertical",
    command=table.yview
)

scroll_x = ttk.Scrollbar(
    records_box,
    orient="horizontal",
    command=table.xview
)

table.configure(
    yscrollcommand=scroll.set,
    xscrollcommand=scroll_x.set
)

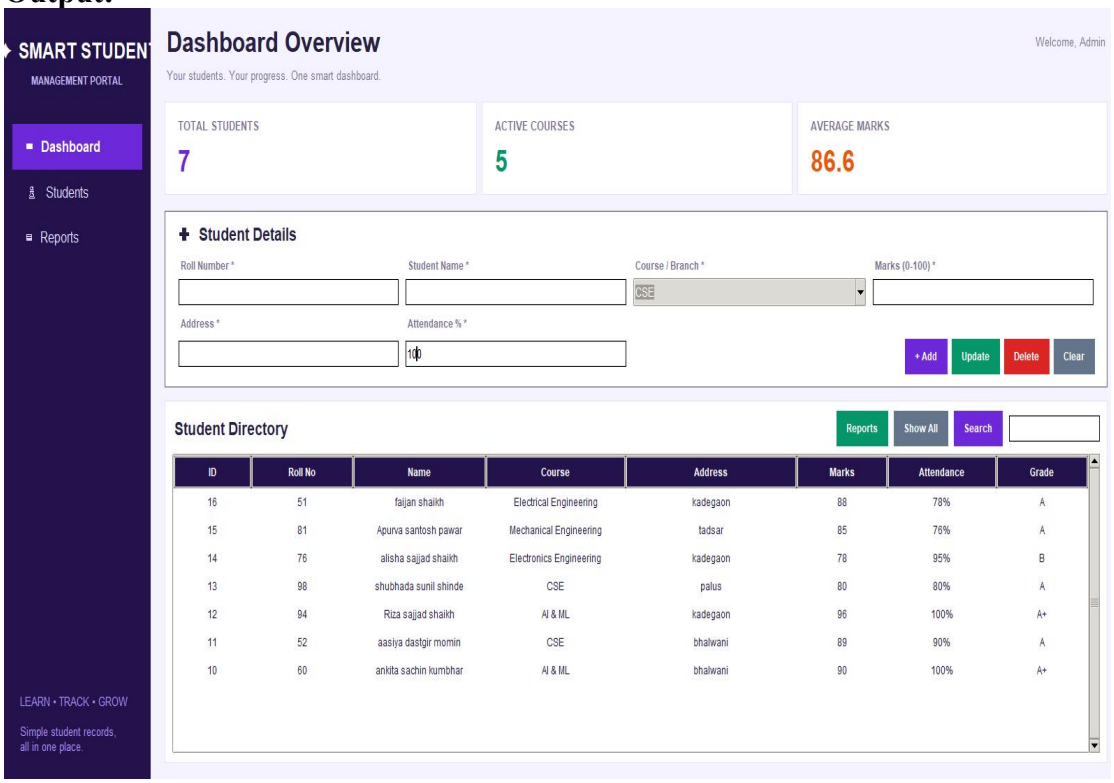
scroll.pack(side="right", fill="y")
table.pack(fill="both", expand=True)
scroll_x.pack(fill="x")

table.bind("<<TreeviewSelect>>", load_form)

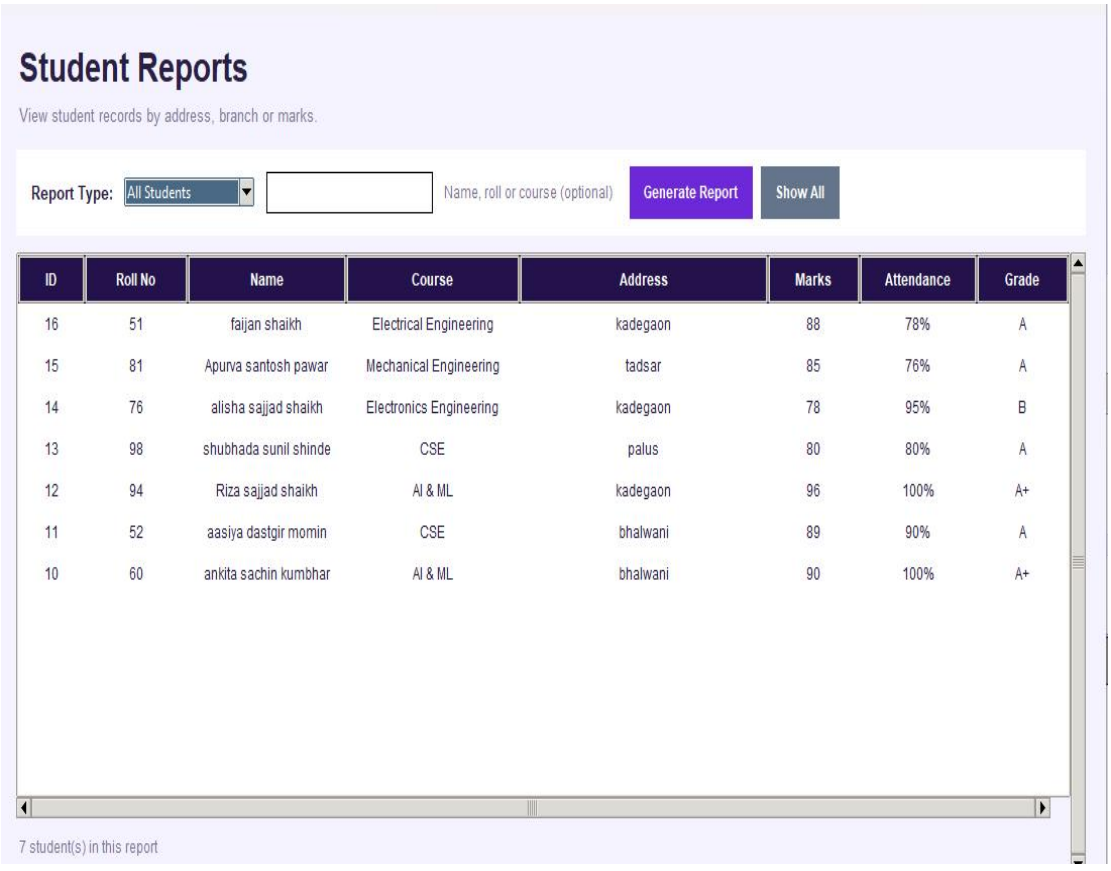
result_label = tk.Label(
    records_box,
    text="0 student(s) displayed",
    bg=WHITE,
    fg=MUTED,
    font=("Arial", 9)
)
result_label.pack(anchor="w", pady=(5, 0))


# ================= START APPLICATION =================
refresh()
root.mainloop()
db.close()
```

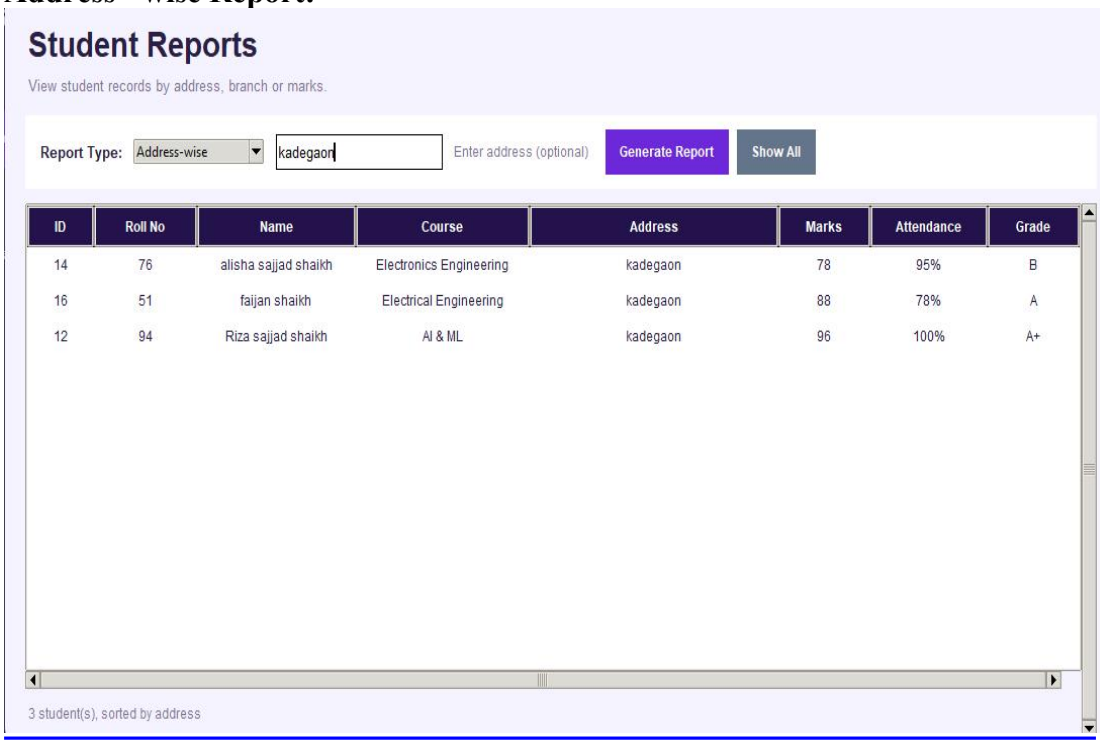
Output:



All Student Report:



Address - wise Report:



Mark -wise Report :

